REDIS: ZERO TO HERO

Beginner-friendly notes • Concepts + commands + Node.js examples + practice

Goal: Understand what Redis is, choose the right data structure, use commands confidently, and build a practical cache with Node.js + Express.

Based on your files: Your syllabus file lists Redis introduction, installation, Strings, Lists, Sets, HashMaps, Sorted Sets, Streams, Geospatial data, other data types, Pub/Sub, and speeding up a Node.js server. Your command log shows Redis Stack in Docker, redis-cli, Strings, NX, MGET, INCR, and expiry. Your JavaScript files use ioredis and Express/axios for a 30-second cache.

How to use these notes

- Read the concept first, then type the commands yourself in redis-cli.
- Run the Node.js examples in a small project rather than only reading them.
- After each chapter, answer the checkpoint questions without looking.
- Redis is fast, but correctness still matters: understand expiry, cache misses, data types, and failure handling.

Learning roadmap

1. Part 1 — Redis fundamentals and local setup
2. Part 2 — Keys, expiry, and Strings
3. Part 3 — Lists, Sets, Hashes, and Sorted Sets
4. Part 4 — Streams, Geospatial, Bitmaps, HyperLogLog, and JSON
5. Part 5 — Pub/Sub and basic messaging
6. Part 6 — Node.js with ioredis
7. Part 7 — Express caching: cache-aside pattern
8. Part 8 — Reliability, security, performance, and practice projects

1. What is Redis?

Redis is an in-memory data store. It keeps data in RAM for very fast reads and writes, and can optionally persist data to disk. It is often used as a cache, but it is also useful for counters, queues, sets, leaderboards, rate limits, sessions, messaging, and event streams.

1.1 Database vs cache

- A primary database (for example PostgreSQL or MongoDB) commonly stores the durable source of truth.
- A cache stores a temporary copy of data that is expensive or slow to fetch again.
- Redis can also be used as a primary data store in some systems, but you must deliberately configure persistence, replication, backups, and recovery.

Analogy: A database is the filing cabinet in another room; Redis is the frequently used notebook on your desk.

1.2 Why Redis is fast

- Most working data is served from memory rather than disk.
- It offers purpose-built data structures and commands such as INCR, LPUSH, SADD, and ZADD.
- A command is usually processed quickly, and pipelining can reduce network round trips.

1.3 Important vocabulary

- Key — a name used to find a value, such as user:42 or cache:products.
- Value — the data stored under a key.
- TTL (time to live) — how long a key has before it expires.
- Cache hit — the requested item exists in cache.
- Cache miss — the requested item is not in cache, so the app fetches it elsewhere.
- Persistence — saving data so it can survive a Redis restart, depending on configuration.
- Eviction — removing keys when memory reaches a configured limit and an eviction policy is active.

2. □ Install and run Redis with Docker

Your commands.txt shows that you started Redis Stack in Docker and mapped Redis port 6379 and the browser UI port 8001.

```
docker run -d --name redis-stack -p 6379:6379 -p 8001:8001 redis/redis-stack:latest
```

Check the container:

```
docker ps
docker logs redis-stack
```

Open a Redis CLI inside the container:

```
docker exec -it redis-stack bash
redis-cli ping
# Expected: PONG
redis-cli
```

The Redis CLI prompt normally looks like 127.0.0.1:6379>. You can also use a Redis GUI at http://localhost:8001 when using the Redis Stack image.

2.1 Useful Docker commands

```
docker stop redis-stack
docker start redis-stack
docker restart redis-stack
docker rm -f redis-stack
```

□ **Important:** Removing a container can remove data stored only inside its writable layer. For data you need to keep, configure a persistent volume and understand the Redis persistence settings.

2.2 Connect from Node.js

Your client.js uses ioredis and creates a client with the default local connection:

```
const { Redis } = require('ioredis');
const client = new Redis();
module.exports = client;
```

Your package.json lists ioredis, express, and axios, with CommonJS enabled.

```
npm init -y
npm install ioredis express axios
```

□ **Connection check:** If Redis is running in Docker with port 6379 published on localhost, a Node.js process on your host can normally connect to localhost:6379. If Node.js itself runs in another container, use the Redis service/container name on a shared Docker network instead of localhost.

3. 🔑 Keys and basic commands

```
PING
SET user:1 "Pradip"
GET user:1
EXISTS user:1
TYPE user:1
DEL user:1
DBSIZE
SCAN 0
```

Naming keys with a colon is a convention, not a Redis requirement. Examples: user:1, session:abc, product:501, cache:todos. Use SCAN for incremental key iteration in applications; avoid relying on KEYS * on a large production database because it can block Redis while scanning all keys.

3.1 Key expiry

```
SET otp:123456 "654321" EX 60
TTL otp:123456
EXPIRE otp:123456 10
PERSIST otp:123456
PTTL otp:123456
```

- EX sets expiry in seconds when writing a key.
- TTL returns remaining whole seconds; -1 means the key exists without an expiry; -2 means the key does not exist.
- PERSIST removes an existing expiry.

💡 **Tip:** Expiry is attached to the key, not to an individual field inside a normal String. Updating a key can affect its expiry depending on the command used, so check TTL when correctness matters.

4. 🔑 Strings — your first Redis data type

A Redis String stores a byte sequence, often used for text, serialized JSON, tokens, simple values, and counters. A Redis String is not limited to human-readable text.

4.1 Core String commands

```
SET name "Pradip Maity"
GET name
MSET user:1 "Pradip" user:2 "Riya"
MGET user:1 user:2
SET nickname "Guddu" NX
SET nickname "NewName" NX
STRLEN name
APPEND name " - IT"
GET name
```

NX means 'only if the key does not already exist'. Your commands.txt demonstrates that a second SET ... NX returns nil when the key already exists. Use XX for 'only if the key already exists'.

4.2 Numeric operations

```
SET views 0
INCR views
INCR views
INCRBY views 10
```

```
DECR views
GET views
```

INCR and INCRBY perform atomic integer updates. This is safer for concurrent counters than reading a value into application code, adding one, and writing it back.

4.3 JSON stored as a String

```
SET user:42 '{"name":"Pradip","role":"student"}'
GET user:42
```

Redis treats that JSON as a String unless you use an additional module/data type such as RedisJSON. In Node.js, use JSON.stringify() before SET and JSON.parse() after GET.

4.4 Node.js example

```
const client = require('./client');

async function main() {
  await client.set('msg:6', 'Hey I am from Node.js');
  await client.expire('msg:6', 10);
  const result = await client.get('msg:6');
  console.log(result);
}

main().catch(console.error);
```

This follows your string.js example. The value is available until the key expires, unless it is overwritten or deleted first.

5. ? Lists — queues and stacks

A Redis List is an ordered sequence of values. You can push or pop from either end. Lists are useful for simple work queues, task buffers, and stack-like behavior.

5.1 Commands

```
LPUSH messages "one" "two" "three"
LRANGE messages 0 -1
RPUSH messages "four"
LLEN messages
LINDEX messages 0
LPOP messages
RPOP messages
```

Redis indexes are zero-based. LRANGE key 0 -1 returns all elements.

5.2 Queue vs stack

- Queue (FIFO): first in, first out. Push on one side and pop from the other, e.g. LPUSH tasks task1 followed by RPOP tasks.
- Stack (LIFO): last in, first out. Push and pop from the same side, e.g. LPUSH stack item followed by LPOP stack.

```
# Queue example
LPUSH jobs "job-1"
LPUSH jobs "job-2"
RPOP jobs
```

Your list.js pushes values with LPUSH and pops using RPOP. That is a queue pattern: oldest item comes out first.

5.3 Blocking queue commands

```
BLPOP jobs 0
BRPOP jobs 0
```

Blocking pops wait until an item becomes available; timeout 0 means wait indefinitely. Use them thoughtfully and ensure your worker has error handling. For modern reliable queues, examine Redis Streams or a dedicated queue library.

Version note: Redis command availability and newer list commands depend on the Redis server version. Check your server version with INFO server or redis-server --version.

6. Sets — unique values

A Set is an unordered collection of unique members. Adding the same member twice does not create a duplicate.

```
SADD course:redis:students "Pradip" "Riya" "Aman"
SADD course:redis:students "Pradip"
SMEMBERS course:redis:students
SISMEMBER course:redis:students "Riya"
SCARD course:redis:students
SREM course:redis:students "Aman"
```

6.1 Set algebra

```
SADD frontend "Pradip" "Riya"
SADD backend "Pradip" "Aman"
SINTER frontend backend
SUNION frontend backend
SDIFF frontend backend
```

Use Sets for tags, unique visitors (when exactness is needed and the set size is manageable), memberships, and relationships such as users who like a post.

Remember: A Set does not promise sorted order. If you need ordering by score or rank, use a Sorted Set.

7. Hashes — field/value records

A Redis Hash stores multiple fields under one key. It resembles a small object or dictionary and is handy for records such as a user profile.

```
HSET user:42 name "Pradip" branch "IT" year 3
HGET user:42 name
HGETALL user:42
HSET user:42 year 4
HINCRBY user:42 year 1
HDEL user:42 branch
HEXISTS user:42 name
```

A Hash can be more convenient than storing each field as a separate key. But do not assume a Redis Hash is identical to a relational database row: it does not automatically enforce schemas, relationships, or constraints.

7.1 String JSON vs Hash

- String with JSON: convenient when you read/write the whole object at once.
- Hash: convenient when you frequently update or read individual fields.

- RedisJSON (when available): supports JSON-oriented operations on nested JSON structures.

8. 🔍 Sorted Sets — rankings and priority

A Sorted Set (ZSET) stores unique members, each associated with a numeric score. Redis orders members by score, making it ideal for leaderboards and ranking.

```
ZADD leaderboard 120 "Pradip" 95 "Riya" 150 "Aman"
ZRANGE leaderboard 0 -1 WITHSCORES
ZREVRANGE leaderboard 0 -1 WITHSCORES
ZSCORE leaderboard "Pradip"
ZINCRBY leaderboard 10 "Riya"
ZREM leaderboard "Aman"
ZCARD leaderboard
```

ZRANGE lists in ascending score order; ZREVRANGE lists in descending score order in the classic command family. Modern Redis also has unified ZRANGE options, depending on version.

8.1 Priority queue pattern

Represent each task as a member and its priority as a score. Decide whether a lower score or higher score means more urgent, then consistently pop the correct end. Sorted Sets provide ranking primitives, but atomic task claiming, retries, and failure recovery require extra design.

 Ties: Members with equal scores are ordered lexicographically. Do not assume insertion order will break ties.

9. 🌊 Streams — event logs and consumer groups

A Redis Stream is an append-only log of entries. Each entry has an ID and fields. Streams are useful for event processing and worker systems where consumers may need to track what they have processed.

```
XADD orders * order_id 501 status created
XADD orders * order_id 502 status paid
XRANGE orders - +
XREVRANGE orders + -
XLEN orders
```

9.1 Consumer groups (concept)

```
XGROUP CREATE orders order-workers 0 MKSTREAM
XREADGROUP GROUP order-workers worker-1 COUNT 2 BLOCK 5000 STREAMS orders >
XACK orders order-workers <entry-id>
```

Replace <entry-id> with the real ID returned by XREADGROUP. Consumer groups distribute work among consumers and keep track of delivered-but-unacknowledged messages. Pending messages may need recovery if a consumer fails.

 Streams vs Pub/Sub: Streams retain entries and support replay/consumer groups. Pub/Sub sends live messages to currently subscribed clients and does not keep a message history for offline subscribers.

10. 🌍 Geospatial commands

Redis geospatial features store longitude/latitude coordinates and support radius/nearby searches. Use longitude first, then latitude.

```
GEOADD stores 88.3639 22.5726 kolkata
GEOADD stores 87.3099 22.6910 haldia
GEOSEARCH stores FROMLONLAT 88.36 22.57 BYRADIUS 50 km WITHDIST
```

Use a meaningful key name such as places or stores. The example coordinates are approximate. Geospatial commands are useful for nearby shops, delivery zones, and location-based discovery, but they are not a full GIS system.

Coordinate order: Redis expects longitude latitude. Swapping them can create wildly incorrect results.

11. Other useful Redis data types

11.1 Bitmaps

Bitmaps operate on bits inside String values. They are useful for compact yes/no flags indexed by an integer, such as whether a user completed a daily action.

```
SETBIT attendance:2026-10-14 42 1
GETBIT attendance:2026-10-14 42
BITCOUNT attendance:2026-10-14
```

11.2 HyperLogLog

HyperLogLog estimates the number of distinct elements using very little memory. It is approximate, so do not use it when exact membership or exact counts are required.

```
PFADD visitors:today user1 user2 user3
PFADD visitors:today user2 user4
PFCOUNT visitors:today
```

11.3 Bitfields

BITFIELD reads or updates small integer fields packed into a String. It is more specialized; learn it after you are comfortable with Strings and bit operations.

11.4 RedisJSON and Redis Stack

Redis Stack distributions may include modules such as RedisJSON and search capabilities, depending on the image/distribution version. These are module features, not the same as the core String/Hash commands. Verify which modules your running server provides.

```
MODULE LIST
```

11.5 Expiring data and memory limits

TTL is not a separate data type, but expiry is a core Redis feature. Redis can also be configured with maxmemory and an eviction policy. Pick policies based on the workload; cache workloads and durable primary data workloads have different requirements.

12. Pub/Sub — live messages

Publish/Subscribe lets a publisher send a message to a channel. Subscribers listening at that moment receive it.

```
# Terminal A
SUBSCRIBE notifications

# Terminal B
PUBLISH notifications "Deployment finished"
```

Pub/Sub is useful for live notifications and lightweight fan-out. It is not a durable queue: a subscriber that is offline misses messages. If the app needs replay, acknowledgements, or durable processing, evaluate Streams.

13. ⚡ Speed up a Node.js server with Redis caching

Your server.js implements the cache-aside pattern for GET /. It first tries client.get('todos'). If the value exists, it parses and returns the cached JSON. If it does not exist, it calls JSONPlaceholder, stores the response for 30 seconds, and returns the data.

13.1 The flow

9. Browser requests GET /.
10. Express asks Redis for the key todos.
11. Cache hit: parse cached JSON and respond without calling the external API.
12. Cache miss: request data from JSONPlaceholder.
13. Store JSON.stringify(data) with an expiry, then respond.

13.2 Your server.js pattern, explained

```
const express = require('express');
const axios = require('axios').default;
const app = express();
const client = require('./client');

app.get('/', async (req, res, next) => {
  try {
    const cachedValue = await client.get('todos');
    if (cachedValue) {
      return res.json(JSON.parse(cachedValue));
    }

    const { data } = await axios.get(
      'https://jsonplaceholder.typicode.com/todos'
    );
    await client.set('todos', JSON.stringify(data), 'EX', 30);
    return res.json(data);
  } catch (err) {
    next(err);
  }
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

This is a slightly hardened version of your file: it wraps asynchronous work in try/catch and passes errors to Express middleware, and it combines SET and expiry into one operation.

13.3 Run and test

```
node server.js
# Open http://localhost:3000 in your browser
# or run:
curl -i http://localhost:3000
```

Try the request twice. The first request should usually call the external API; the second should be served from Redis while the key is alive. You can inspect the key with redis-cli GET todos and TTL todos.

13.4 Cache pitfalls

- Cache staleness: cached data may be out of date until TTL expires or you invalidate it.
- Cache stampede: many simultaneous misses may all call the upstream API. Mitigations include request coalescing, locks used carefully, jittered TTLs, or stale-while-revalidate.
- Redis outage: decide whether the route should fail, fall back to the source, or serve stale data.
- Large payloads: JSON serialization and memory usage have costs. Cache only what provides a useful speed/availability trade-off.
- Key collisions: use namespaced keys, e.g. cache:todos:v1.
- User-specific data: include relevant user/tenant/locale/permissions in the cache key to avoid serving one user's data to another.
- Error handling: external API failures and invalid cached JSON should not crash the entire process without a deliberate policy.

13.5 Measure the difference

Use timing logs or a benchmark to compare cache hits and misses. Avoid claiming a fixed speedup without measuring your own app and workload.

14. ⍰ Reliability, security, and performance

14.1 Persistence

- RDB snapshots save point-in-time snapshots.
- AOF (Append Only File) records write operations for recovery.
- Some deployments combine them. Persistence choices affect durability, disk use, and recovery time.

14.2 Replication and high availability

Replication can maintain replicas. Sentinel can monitor Redis and help with failover in supported setups. Redis Cluster partitions data across nodes for scale. These are deployment topics; a single local Docker container does not automatically provide high availability.

14.3 Security basics

- Do not expose Redis publicly to the internet.
- Use network isolation, authentication/ACLs, and TLS where appropriate.
- Use least-privilege users and avoid putting passwords in source code.
- Keep Redis and container images patched.

14.4 Performance habits

- Prefer a small number of appropriate commands over repeated network round trips.
- Use pipelining for batches when it helps; pipelining is not the same as a transaction.
- Avoid huge blocking operations and very large keys.
- Monitor memory, latency, key expiry, hit rate, and evictions.
- Use realistic key names and set TTLs for cache entries.

14.5 Atomicity, transactions, and Lua

A single Redis command such as INCR is atomic. MULTI/EXEC groups commands into a transaction-like execution block, while WATCH can support optimistic concurrency. Lua scripts can run server-side logic atomically. These features are useful when several operations must behave as one logical update.

```
MULTI
SET order:501 status:paid
INCR orders:paid:count
EXEC
```

 Important: Redis transactions do not work exactly like SQL transactions: they do not provide general rollback of earlier successful commands if another queued command has a runtime error.

15. ? Quick data-structure chooser

Need	Use	Example
One value or counter	String	page:view count
Object-like fields	Hash	user profile
Unique members	Set	tags or membership
Ranked by score	Sorted Set	leaderboard
Push/pop sequence	List	simple queue/stack
Durable event log / consumer groups	Stream	order events
Live fan-out messages	Pub/Sub	live notification
Nearby coordinates	Geospatial	stores near user
Compact bit flags	Bitmap	daily attendance
Approximate distinct count	HyperLogLog	unique visitors estimate

16. ? Practice lab: do these yourself

14. Create a key profile:name and store your name. Read it back.
15. Create a key otp:demo with a 20-second expiry. Check TTL before and after waiting.
16. Create a counter and increase it 100 times using INCR.
17. Build a queue using LPUSH and RPOP. Prove that it is FIFO.
18. Create two Sets for students in frontend and backend. Find common and unique students.
19. Store a user record in a Hash and update only the year field.
20. Create a leaderboard and return the top three players.
21. Write three events to a Stream and read them back.
22. Publish a notification from one terminal and subscribe in another.
23. Add a Redis cache to an Express route and confirm that repeated requests reuse the cached value.

17. ? Interview / revision questions

24. What is Redis, and why is it fast?
25. Is Redis only a cache? Explain.
26. What is a cache hit versus a cache miss?
27. What does TTL return when a key has no expiry or does not exist?
28. What is the difference between SET key value NX and a normal SET?
29. Why is INCR safer than GET, increment in JavaScript, then SET?
30. How do Redis Lists implement a queue and a stack?
31. How is a Set different from a Sorted Set?
32. When would you choose a Hash over a JSON String?
33. What is the difference between Streams and Pub/Sub?

34. What is cache-aside, and what are its failure modes?
35. What do RDB and AOF do?
36. Why should you avoid exposing Redis directly to the public internet?
37. What are pipelining, transactions, and consumer groups?
38. How would you prevent one user's cached response from being served to another user?

18. 🧭 Recommended learning order

39. Master PING, SET, GET, DEL, TTL, EXPIRE, MGET, INCR, and SET ... NX.
40. Practice Lists, Sets, Hashes, and Sorted Sets with small examples.
41. Build the ioredis client and learn async/await, error handling, and connection lifecycle.
42. Implement the Express cache-aside example and test hit/miss behavior.
43. Learn Streams and Pub/Sub, then decide which fits a real messaging problem.
44. Explore persistence, eviction policies, monitoring, security, and deployment only after the basics feel comfortable.

19. 🧾 Cheat sheet

```
PING                        # health check
SET key value               # write a String
GET key                     # read a String
SET key value EX 60         # write with 60-second TTL
TTL key                     # remaining seconds
DEL key                     # delete key
MGET key1 key2              # read multiple String keys
INCR counter                # atomic increment
LPUSH list value            # push to list's left
RPOP list                   # pop from list's right
SADD set member             # add unique member
SMEMBERS set                # list Set members
HSET hash field value       # set Hash field
HGETALL hash                # read all Hash fields
ZADD zset score member      # add Sorted Set member
ZRANGE zset 0 -1 WITHSCORES # sorted members + scores
XADD stream * field value   # append Stream event
PUBLISH channel message     # publish live message
GEOADD places lon lat name  # add location
```

Final reminder

Redis becomes much easier when you connect each data type to a problem: Strings for values/counters, Lists for sequences, Sets for uniqueness, Hashes for fields, Sorted Sets for ranking, Streams for retained events, and Pub/Sub for live delivery. Start with the commands in your own commands.txt, then build on your existing ioredis + Express project.